

Universidad de Buenos Aires
Maestría en Inteligencia Artificial

Trabajo Práctico Final

Aprendizaje por Refuerzo II

Control Adaptativo de Tráfico mediante PPO-GAE,
A2C y Curriculum Learning

Autores:

Jorge Cuenca
Fabian Sarmiento
Gustavo Rivas

Docente:

Esp. Ing. Miguel Augusto Azar

Fecha:

Abril 2026

Índice general

1. Resumen Ejecutivo	1
1.1. 1.1 Objetivo, hipótesis y criterios de evaluación	1
1.2. 1.2 Alcance implementado y alineación con el repositorio	2
2. Descripción del Problema y Enfoque	3
2.1. 2.1 Formulación del Problema	3
2.2. 2.2 Problema Principal: Control Adaptativo de Semáforos en Grilla 4×4	3
2.3. 2.3 Observación, Acción y Espacio de Estado	4
2.4. 2.4 Visualización del Modelo de Semáforos	4
3. Algoritmos Implementados	7
3.1. 3.1 A2C — Advantage Actor-Critic (Baseline, Fase 2)	7
3.2. 3.2 PPO-GAE — Proximal Policy Optimization con GAE (Fase 3)	8
3.3. 3.3 Las 8 Mejoras Algorítmicas de PPO-GAE sobre A2C	9
3.4. 3.4 Arquitectura de Red Neuronal — SharedActorCritic	10
3.5. 3.5 Ciclo de Aprendizaje Actor-Crítico	11
4. Resumen de Técnicas y Configuración Experimental	12
4.1. 4.1 Hiperparámetros Principales	13
4.2. 4.2 Curriculum Learning (Fase 5)	13
5. Resultados y Gráficos de Entrenamiento	15
5.1. 5.1 Curvas de Entrenamiento — PPO-GAE (6 métricas)	15
5.2. 5.2 Curvas de Entrenamiento — A2C (Baseline, Fase 2)	16
5.3. 5.3 Comparación A2C vs PPO-GAE	17
5.4. 5.4 Comparación contra Controladores de Referencia (Fase 4)	17
5.5. 5.5 Distribución multi-semilla PPO-GAE	19
5.6. 5.6 Comparación Dinámica Paso a Paso (Fase 4)	19
5.7. 5.7 Interpretación de la Política Aprendida	19
6. Inconvenientes Encontrados y Soluciones	21
6.1. 6.1 Inestabilidad de A2C en Espacio de Acción de Alta Dimensión	21

6.2.	6.2 Cold-Start Problem con Demanda de Tráfico Completa	21
6.3.	6.3 Normalización Incorrecta del Reward (Bug F1/F2/F3 en v2)	22
6.4.	6.4 Acción MultiDiscrete y Compatibilidad con VecEnv	23
6.5.	6.5 Dependencia Opcional de SUMO	23
7.	Arquitectura del Proyecto	24
7.1.	7.1 Estructura General	24
7.2.	7.2 Módulos Clave	25
7.3.	7.3 Reproducibilidad y Semillas	25
7.4.	7.4 Comandos de Ejecución	26
8.	Conclusiones, Limitaciones y Trabajo Futuro	28
8.1.	8.1 Limitaciones del Trabajo	29
8.2.	8.2 Trabajo Futuro	29
9.	Referencias	30

Índice de figuras

1.	Visualización estática del modelo de semáforos: grilla 4×4 con fases activas y barras de cola por dirección.	5
2.	Matriz de estado observado por el agente: 16 intersecciones × 9 características.	6
3.	Diagrama conceptual del ciclo de aprendizaje actor-crítico para control adaptativo de semáforos.	11
4.	Progresión de demanda del curriculum learning.	14
5.	Curvas de entrenamiento PPO-GAE: policy loss, value loss, entropía, KL divergence, explained variance y reward por update.	15
6.	Curvas de entrenamiento A2C: policy loss, value loss, entropía y reward por update.	16
7.	Comparación directa de curvas de entrenamiento A2C vs. PPO-GAE.	17
8.	Comparación de reward medio con barras de error para los cinco controladores.	18
9.	Comparación de longitud media de colas vehiculares.	18
10.	Distribución del reward de PPO-GAE sobre 5 semillas × 20 episodios.	19
11.	Mapa de calor comparativo de dirección con mayor cola y fase elegida por A2C y PPO-GAE.	20

Índice de cuadros

1.	Mejoras algorítmicas de PPO-GAE sobre A2C	9
2.	Fases experimentales y configuración general.	12
3.	Hiperparámetros principales de A2C y PPO-GAE.	13
4.	Etapas del curriculum learning.	14
5.	Comparación final contra controladores de referencia	17

Capítulo 1

Resumen Ejecutivo

El presente trabajo corresponde al Trabajo Práctico Final de la materia Aprendizaje por Refuerzo II de la Maestría en Inteligencia Artificial (UBA, 2026). El objetivo fue diseñar e implementar un agente de aprendizaje por refuerzo profundo capaz de controlar de forma adaptativa los semáforos de una grilla de intersecciones, minimizando las colas vehiculares y los tiempos de espera.

Se desarrolló un entorno de simulación de tráfico completamente en Python sin dependencias externas de simuladores, y se compararon dos algoritmos Actor-Critic: A2C (Advantage Actor-Critic) como baseline y PPO-GAE (Proximal Policy Optimization con Generalized Advantage Estimation) como implementación avanzada. PPO-GAE incorpora 8 mejoras algorítmicas sustanciales sobre A2C y supera a todos los controladores de referencia evaluados.

1.1. 1.1 Objetivo, hipótesis y criterios de evaluación

El objetivo del trabajo no es solo implementar algoritmos Actor-Critic, sino demostrar empíricamente que las mejoras introducidas por PPO-GAE resuelven las limitaciones prácticas de A2C en un problema de control multiagente con espacio de estado y acción de alta dimensión.

A partir de este objetivo se plantean las siguientes hipótesis de trabajo:

- PPO-GAE converge a una política estable y de alta calidad que A2C no logra alcanzar en el mismo número de iteraciones.
- El clipping de la función objetivo PPO previene los colapsos de política que generan la alta varianza observada en A2C.
- La estimación GAE reduce la varianza de los gradientes frente a los retornos de n pasos utilizados por A2C.
- El Curriculum Learning mejora la eficiencia de entrenamiento al graduar la dificultad del entorno desde demanda baja (30 %) hasta demanda completa (100 %).

Como criterios de evaluación se consideran: la recompensa media por episodio sobre múltiples semillas, la varianza de esa recompensa (estabilidad), la longitud media de colas vehiculares y el tiempo de espera promedio. Estos valores se comparan contra cinco políticas de referencia: Random, Fixed-Time, Actuated, A2C y PPO-GAE.

1.2. 1.2 Alcance implementado y alineación con el repositorio

El repositorio implementa los cinco fases del plan de trabajo propuesto. La Fase 1 construye el entorno custom MultiIntersectionEnv (grilla 4×4, 144 dimensiones de observación, espacio de acción MultiDiscrete). La Fase 2 entrena el agente A2C baseline. La Fase 3 implementa PPO-GAE con 8 mejoras algorítmicas documentadas. La Fase 4 evalúa contra 5 controladores de referencia y realiza un barrido de hiperparámetros. La Fase 5 añade Curriculum Learning, exportación TorchScript y monitoreo con TensorBoard.

El notebook `tp_final_notebook.ipynb` integra todas las fases en un único flujo reproducible, mientras que los scripts individuales (`train_ppo.py`, `train_a2c.py`, `evaluate.py`, `sweep.py`) permiten ejecutar cada fase de forma independiente.

Capítulo 2

Descripción del Problema y Enfoque

2.1. 2.1 Formulación del Problema

El Aprendizaje por Refuerzo modela la interacción de un agente con su entorno mediante el paradigma del Proceso de Decisión de Markov (MDP), definido como la tupla (S, A, P, R, γ) , donde S es el espacio de estados, A el espacio de acciones, P la función de transición, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento. El objetivo es aprender una política π que maximice el retorno esperado acumulado $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$.

En este trabajo el problema de control de tráfico se formula como un MDP con espacio de estado continuo de alta dimensión ($\mathbb{R}^{144}$) y espacio de acción combinatorio ($\text{MultiDiscrete}([4]^{16})$). Esta complejidad hace inviables los métodos tabulares clásicos de RLI y requiere aproximación por función neuronal profunda con algoritmos de política de gradiente.

2.2. 2.2 Problema Principal: Control Adaptativo de Semáforos en Grilla 4×4

El entorno central es `MultiIntersectionEnv`, una grilla de 16 intersecciones (4 filas $\times$ 4 columnas) implementada íntegramente en Python. Cada intersección tiene 4 carriles y puede estar en uno de 4 fases de semáforo. Un agente centralizado observa el estado completo de la grilla y selecciona la fase activa de cada intersección en cada paso de tiempo.

La dinámica de vehículos sigue un proceso de Poisson independiente por carril ($\lambda = 0,3$ veh/paso a demanda 1.0). Las partidas son aproximadamente 2 vehículos/paso desde los carriles servidos por la fase activa. Se impone una restricción de verde mínimo de 5 pasos antes de permitir un cambio de fase. Cada episodio tiene 720 pasos (~ 1 hora simulada a 5 segundos/paso).

La señal de recompensa densa penaliza simultáneamente las colas y los tiempos de espera:

$$r_t = -\frac{\sum_i \text{queue_lengths}_i}{N_{\text{lanes}}} - 0,1 \frac{\sum_i \text{waiting_times}_i}{N_{\text{lanes}}}. \quad (2.1)$$

Esta señal es negativa en todo momento (problema de minimización); valores cercanos a cero indican tráfico fluido, mientras que valores muy negativos reflejan congestión severa.

2.3. 2.3 Observación, Acción y Espacio de Estado

Vector de observación (144 dimensiones): por cada una de las 16 intersecciones se extraen 9 características — longitud de cola en 4 carriles (normalizada por capacidad de 20 vehículos), fase actual (codificación one-hot de 4 dimensiones) y tiempo transcurrido en la fase actual (normalizado por 120 segundos). El espacio resultante es $[0, 1]^{144}$.

Espacio de acción: MultiDiscrete($[4]^{16}$), es decir 16 decisiones discretas independientes (una por intersección), cada una seleccionando la próxima fase activa entre 4 posibles. El número total de combinaciones de acción es $4^{16} \approx 4,3 \times 10^9$, lo que hace inviable cualquier enfoque tabular.

2.4. 2.4 Visualización del Modelo de Semáforos

El notebook incluye una representación visual del entorno que permite interpretar el comportamiento del sistema antes de entrenar los agentes. La grilla 4×4 muestra las 16 intersecciones con sus fases activas de semáforo, las colas de vehículos por dirección y la dinámica del tráfico en tiempo real. Esta visualización no modifica el entrenamiento: sirve para comprender qué está aprendiendo el agente.

En cada intersección se muestran: la fase activa del semáforo (indicada con una luz verde), las colas de vehículos por dirección (Norte, Este, Sur, Oeste) representadas como barras proporcionales, y el comportamiento dinámico del entorno a medida que cambian las fases. El notebook también genera animaciones HTML compatibles con Google Colab que permiten observar la evolución del tráfico paso a paso.

Modelo visual del entorno de semáforos 4×4

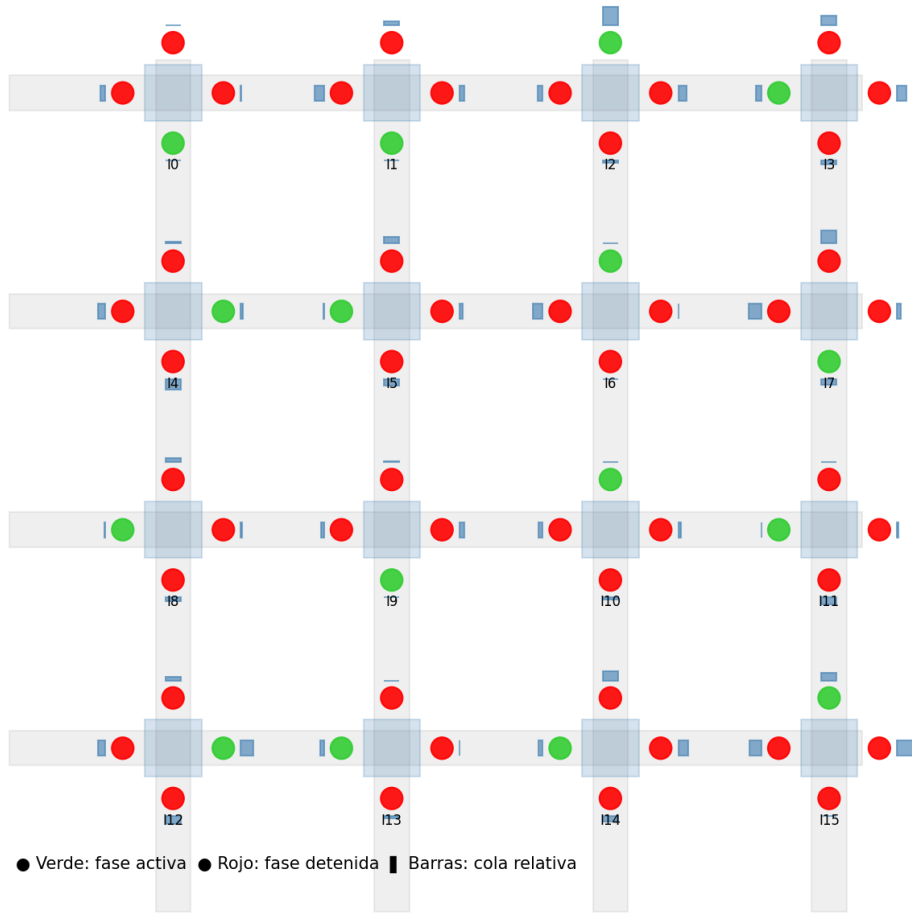


Figura 1: Visualización estática del modelo de semáforos: grilla 4×4 con fases activas y barras de cola por dirección.

El estado observado por el agente se representa además como una matriz 16×9, donde cada fila corresponde a una intersección y cada columna a una de las nueve características: cola en las cuatro direcciones (normalizada), fase activa en codificación one-hot (4 dimensiones) y tiempo de simulación normalizado. Esta representación facilita verificar que el agente recibe información significativa y no artefactos numéricos.

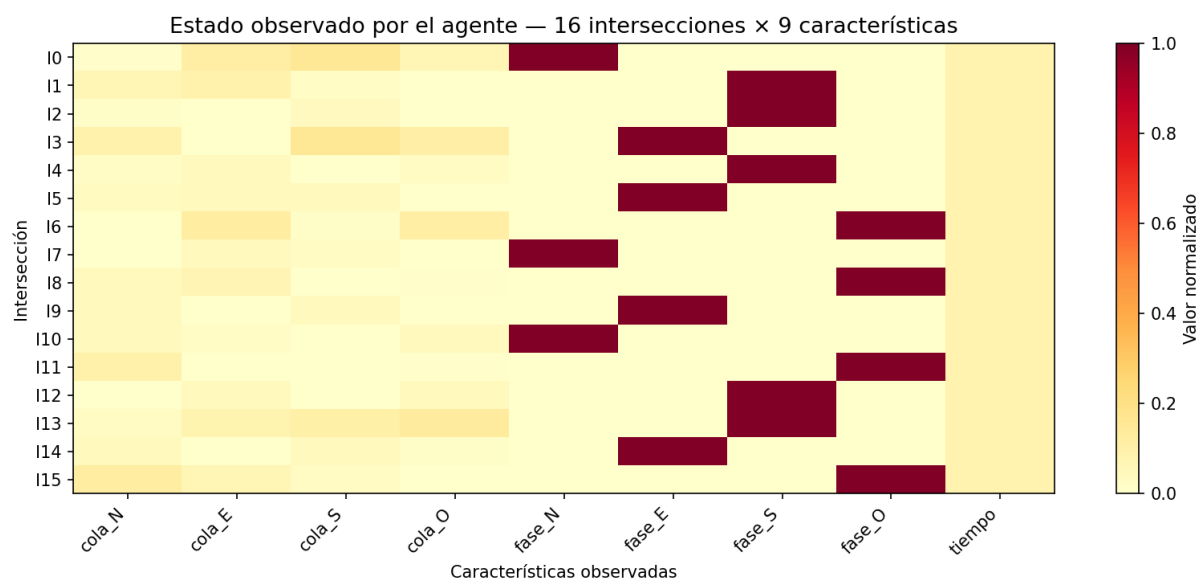


Figura 2: Matriz de estado observado por el agente: 16 intersecciones × 9 características.

Capítulo 3

Algoritmos Implementados

3.1. 3.1 A2C — Advantage Actor-Critic (Baseline, Fase 2)

A2C es un algoritmo Actor-Critic on-policy y síncrono. El actor aprende una política estocástica $\pi_\theta(a|s)$, mientras que el crítico estima la función de valor $V_\phi(s)$ para calcular la ventaja que guía las actualizaciones del actor. A diferencia de A3C (versión asíncrona), A2C sincroniza todos los entornos antes de cada actualización del gradiente.

La función de pérdida combinada es:

$$L = -L_{\text{actor}} - c_e H(\pi_\theta) + c_v L_{\text{critic}}, \quad (3.1)$$

$$L_{\text{actor}} = \mathbb{E}[\log \pi_\theta(a | s) A_t], \quad (3.2)$$

$$L_{\text{critic}} = \text{MSE}(V_\phi(s), G_t), \quad (3.3)$$

$$H(\pi_\theta) = - \sum_a \pi_\theta(a | s) \log \pi_\theta(a | s). \quad (3.4)$$

Las ventajas se calculan como retornos de n pasos (n-step returns) sin GAE. La implementación utiliza 16 entornos vectorizados en paralelo (SyncVectorEnv), una única época de actualización por rollout, y coeficiente de entropía estático $c_e = 0.01$.

Listing 3.1: Actualización A2C con retornos n-step.

```

1 # Actualización A2C -- n-step returns (sin GAE)
2 advantages = returns - values.detach()
3 loss_actor = -(log_probs * advantages.detach()).mean()
4 loss_critic = F.mse_loss(values, returns)
5 loss_entropy = dist_entropy.mean()
6
7 loss = loss_actor + c_v * loss_critic - c_e * loss_entropy
8 loss.backward()
9 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)
10 optimizer.step()

```

Limitación observada: A2C mostró alta inestabilidad en este problema (std = 303.62 en evaluación), lo que motivó el desarrollo de PPO-GAE con mejoras específicas para estabilizar el entrenamiento.

3.2. 3.2 PPO-GAE — Proximal Policy Optimization con GAE (Fase 3)

PPO (Schulman et al., 2017) introduce un mecanismo de clipping en la función objetivo del actor que limita el tamaño de cada actualización de política, previniendo los colapsos que sufre A2C. La función de pérdida clipeada es:

$$L_{\text{CLIP}}(\theta) = \mathbb{E}[\text{mín}(r_t A_t, \text{clip}(r_t, 1 - \varepsilon, 1 + \varepsilon) A_t)], \quad (3.5)$$

$$r_t = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \quad \varepsilon = 0.2. \quad (3.6)$$

La Estimación de Ventaja Generalizada (GAE, Schulman et al., 2015) usa un parámetro λ para interpolar entre retornos de un paso (alta sesgo, baja varianza) y retornos de Monte Carlo (bajo sesgo, alta varianza):

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (3.7)$$

$$A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}. \quad (3.8)$$

Listing 3.2: Implementación backward pass de GAE.

```

1 gae = 0
2 for t in reversed(range(T)):
3     delta = rewards[t] + gamma * values[t + 1] * (1 - dones[t]) - values[t]
4     gae = delta + gamma * lam * (1 - dones[t]) * gae
5     advantages[t] = gae
6 returns = advantages + values[:-1]

```

3.3. 3.3 Las 8 Mejoras Algorítmicas de PPO-GAE sobre A2C

La siguiente tabla resume las 8 mejoras implementadas en PPO-GAE respecto al baseline A2C:

Cuadro 1: Mejoras algorítmicas de PPO-GAE sobre A2C

#	Mejora	Descripción	Impacto
1	PPO Clipping ($\epsilon=0.2$)	Acota las actualizaciones de política mediante ratio clipeado	Previene colapsos de política
2	GAE ($\lambda=0.95$)	Estimación de ventaja exponencialmente ponderada	Reduce varianza de gradiente vs n-step
3	Reward Normalization	RunningMeanStd (algoritmo de Welford) online	Estabiliza la señal de recompensa
4	Gradient Clipping (0.5)	Norma máxima del gradiente en actor y crítico	Previene divergencia de parámetros
5	Entropy Decay ($0.05 \rightarrow 0.01$)	Decaimiento lineal del coeficiente de entropía	Explora más al inicio, explota al final
6	Multi-Epoch Updates ($K=4$)	4 épocas PPO por rollout con mini-batches	Reutiliza $4\times$ cada experiencia
7	Optimizadores separados	Actor $lr=3e-4$ vs Crítico $lr=1e-3$	El crítico aprende más rápido
8	Curriculum Learning	3 etapas de demanda: $30\% \rightarrow 60\% \rightarrow 100\%$	Mitiga el cold-start problem

Listing 3.3: Pérdida PPO completa con early stopping por KL.

```

1  # Pérdida PPO completa
2  loss_clip = -torch.min(ratio * adv, clipped_ratio * adv).mean()
3  loss_vf = F.mse_loss(values, returns) # + VF clipping PPO-style
4  loss_entropy = -dist_entropy.mean()
5
6  loss = loss_clip + c_v * loss_vf + c_e * loss_entropy
7
8  # KL Early Stopping: abortar epoch si KL > kl_target
9  approx_kl = (old_log_probs - new_log_probs).mean().item()
10 if approx_kl > kl_target:
11     break

```

3.4. 3.4 Arquitectura de Red Neuronal — SharedActorCritic

Ambos algoritmos comparten la misma arquitectura de red neuronal (models/actor_critic.py). El tronco compartido extrae características del estado; a partir de él se bifurcan las 16 cabezas del actor (una por intersección) y la cabeza del crítico:

Listing 3.4: Arquitectura compartida actor-crítico.

```

1  class SharedActorCritic(nn.Module):
2      # Tronco compartido: 144 -> 256 -> 256
3      self.trunk = nn.Sequential(
4          nn.Linear(144, 256), nn.ReLU(),
5          nn.Linear(256, 256), nn.ReLU()
6      )
7      # 16 cabezas independientes del actor (una por intersección)
8      self.actor_heads = nn.ModuleList([
9          nn.Linear(256, 4) for _ in range(16)
10     ])
11     # Cabeza del crítico: 256 -> 128 -> 1
12     self.critic = nn.Sequential(
13         nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 1)
14     )
15
16     def forward(self, obs):
17         shared = self.trunk(obs) # R^256
18         dists = [Categorical(logits=h(shared)) for h in self.actor_heads]
19         value = self.critic(shared).squeeze(-1) # escalar V(s)
20         return dists, value

```

Parámetros totales: ~180K. Inicialización ortogonal con ganancias diferenciadas para actor

(0.01) y crítico (1.0). El forward pass produce 16 distribuciones categóricas independientes sobre las que se muestrea una acción por intersección.

3.5. 3.5 Ciclo de Aprendizaje Actor-Crítico

Ambos métodos (A2C y PPO-GAE) pertenecen a la familia actor-crítico. Esto significa que el modelo aprende dos funciones en paralelo: el actor decide qué acción tomar (qué fase de semáforo activar en cada intersección), mientras que el crítico estima el valor esperado del estado actual, es decir, cuánta recompensa acumulada puede esperarse desde ese punto.

El ciclo de aprendizaje sigue los siguientes pasos: (1) el agente recibe el estado del entorno (colas + fases + tiempo); (2) el actor selecciona una acción (fase por intersección); (3) el entorno simula el tráfico y devuelve una recompensa; (4) el crítico estima el valor $V(s)$; (5) se calcula la ventaja $A(s,a)$ que mide cuánto mejor o peor fue la acción respecto a lo esperado; (6) tanto el actor como el crítico se actualizan usando el gradiente de sus respectivas funciones de pérdida. El nuevo estado resultante reinicia el ciclo.

La diferencia principal entre A2C y PPO-GAE radica en el paso de actualización: A2C aplica el gradiente de política directamente sobre la ventaja estimada, lo que puede generar actualizaciones bruscas en espacios de acción de alta dimensión. PPO introduce un término de clipping que impide que la nueva política se aleje demasiado de la anterior, mientras que GAE suaviza la estimación de ventaja combinando retornos de distintos horizontes temporales para equilibrar sesgo y varianza.

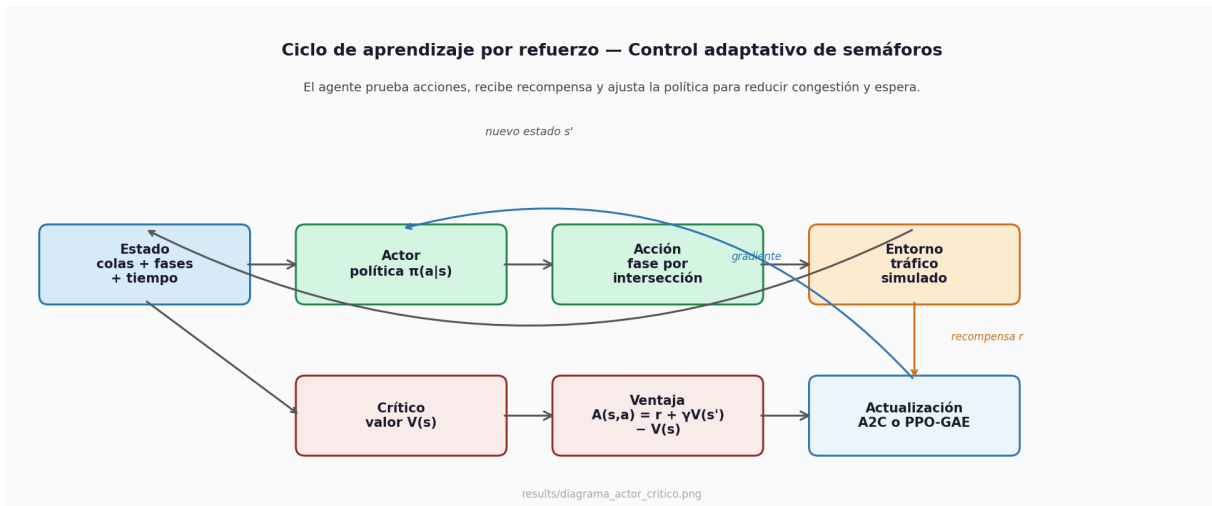


Figura 3: Diagrama conceptual del ciclo de aprendizaje actor-crítico para control adaptativo de semáforos.

Capítulo 4

Resumen de Técnicas y Configuración Experimental

Este capítulo resume el diseño experimental utilizado para comparar las distintas técnicas de aprendizaje por refuerzo implementadas en el proyecto. El objetivo es dejar explícitas las condiciones bajo las cuales se entrenaron y evaluaron los agentes, de modo que los resultados posteriores puedan interpretarse de forma reproducible y comparable. En particular, se distinguen las fases de entrenamiento base, las mejoras incorporadas con PPO-GAE, la evaluación multi-semilla, el barrido de hiperparámetros y la variante con curriculum learning.

La Tabla siguiente presenta una vista general de las fases experimentales. Para cada fase se indica el algoritmo o procedimiento utilizado, el entorno de simulación, la cantidad de actualizaciones de entrenamiento y el número de entornos paralelos empleados. Esta síntesis permite ubicar rápidamente qué rol cumple cada experimento dentro del flujo completo del trabajo: A2C actúa como línea base, PPO-GAE como método principal, la evaluación multi-semilla mide robustez, el sweep explora sensibilidad a hiperparámetros y el curriculum learning analiza el efecto de introducir dificultad progresiva.

Cuadro 2: Fases experimentales y configuración general.

Fase	Algoritmo	Entorno	Updates	Envs
Fase 2	A2C (baseline)	Grilla 4×4	200	16
Fase 3	PPO-GAE	Grilla 4×4	500	16
Fase 4	Eval. multi-semilla	Grilla 4×4	5 seeds × 20 eps	—
Fase 4	Sweep	Grilla 4×4	100/config	—
Fase 5	PPO + Curriculum	Grilla 4×4	500	16

4.1. 4.1 Hiperparámetros Principales

Además de definir las fases, resulta necesario detallar los hiperparámetros que condicionan el comportamiento de cada algoritmo. Estos valores controlan aspectos centrales del aprendizaje, como la ponderación de recompensas futuras, la estabilidad de las actualizaciones, el tamaño de los rollouts, la exploración inducida por entropía y la magnitud máxima de los gradientes. Mantenerlos explícitos facilita la reproducción de los entrenamientos y permite explicar diferencias observadas entre A2C y PPO-GAE.

La Tabla de hiperparámetros compara la configuración usada para el baseline A2C y para PPO-GAE. En A2C se emplean retornos de n pasos y una actualización más directa, mientras que PPO-GAE incorpora ventajas generalizadas, clipping de política, múltiples épocas por rollout, mini-batches y mecanismos adicionales de estabilización como early stopping por KL y decaimiento de learning rate. Por este motivo, la tabla no solo documenta valores numéricos, sino que también muestra las diferencias metodológicas entre ambos enfoques.

Cuadro 3: Hiperparámetros principales de A2C y PPO-GAE.

Parámetro	A2C	PPO-GAE
Factor de descuento γ	0.99	0.99
GAE lambda λ	— (n-step)	0.95
PPO clip ε	— (sin clip)	0.2
Learning rate actor	3e-4	3e-4 (decay lineal)
Learning rate crítico	3e-4 (shared)	1e-3 (separado)
Épocas PPO por rollout	1	4
Mini-batch size	—	256
Rollout length N	128	128
Coef. entropía c_e	0.01 (estático)	0.05→0.01 (decay)
Coef. valor c_v	0.5	0.5
Grad clip max norm	0.5	0.5
KL target (early stop)	—	0.02
Warmup steps	—	512 (pre-seed normalizer)

4.2. 4.2 Curriculum Learning (Fase 5)

La Fase 5 incorpora un programador de curriculum (utils/curriculum.py) que escala progresivamente la demanda de tráfico en tres etapas para evitar el cold-start problem:

La motivación de esta estrategia es reducir la dificultad inicial del entorno y permitir que el agente aprenda primero comportamientos básicos de control semafórico antes de enfrentarse a la demanda completa. En un escenario de tráfico con múltiples intersecciones, comenzar

directamente con alta congestión puede producir recompensas muy negativas y señales de aprendizaje poco informativas. El curriculum busca suavizar esa transición mediante niveles de demanda crecientes.

La Tabla correspondiente describe las tres etapas del curriculum learning. Cada fila especifica el porcentaje de demanda usado, la duración prevista de la etapa y el criterio de avance hacia el siguiente nivel. De esta forma, el entrenamiento no queda definido únicamente por un número fijo de updates, sino también por el desempeño promedio alcanzado por el agente durante la fase actual.

Cuadro 4: Etapas del curriculum learning.

Etapas	Demanda	Duración	Umbral de avance
Etapa 1	30 %	150 updates	reward promedio < -5.5
Etapa 2	60 %	150 updates	reward promedio < -3.5
Etapa 3	100 %	remaining updates	—

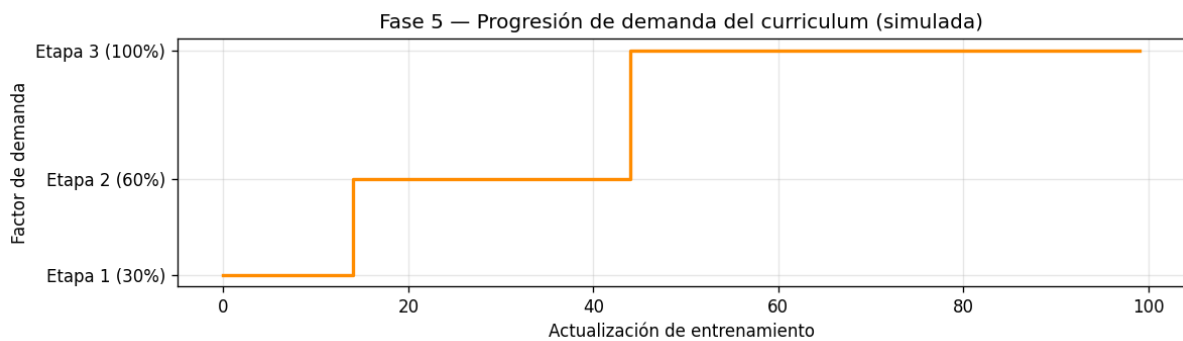


Figura 4: Progresión de demanda del curriculum learning.

Capítulo 5

Resultados y Gráficos de Entrenamiento

5.1. 5.1 Curvas de Entrenamiento — PPO-GAE (6 métricas)

Los gráficos de entrenamiento PPO-GAE muestran la evolución de seis métricas simultáneas a lo largo de 500 iteraciones de actualización:

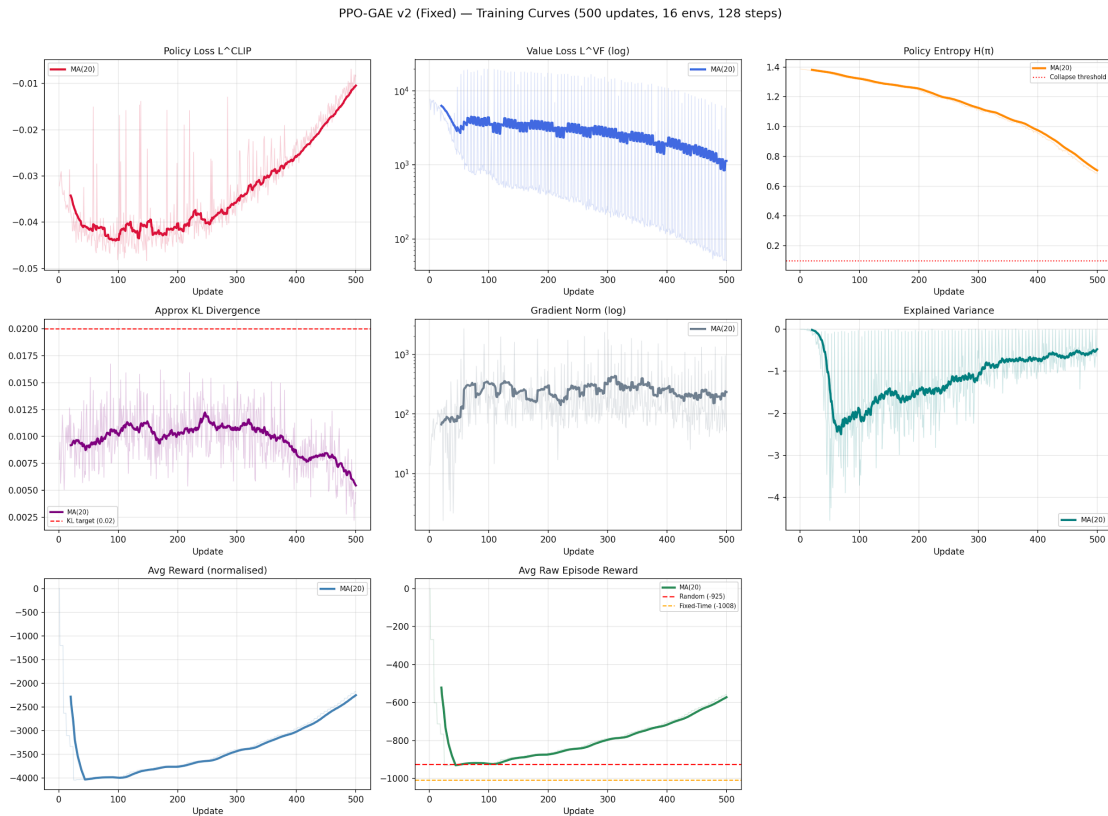


Figura 5: Curvas de entrenamiento PPO-GAE: policy loss, value loss, entropía, KL divergence, explained variance y reward por update.

Descripción de las curvas observadas:

- **Policy Loss**: desciende progresivamente a lo largo del entrenamiento, con oscilaciones contenidas por el clipping PPO ($\epsilon=0.2$).

- Value Loss: disminuye indicando que el crítico mejora su estimación de $V(s)$; la curva se estabiliza alrededor del update 300.
- Entropía: decae linealmente de 0.05 a 0.01, siguiendo el schedule programado. Exploración alta en las primeras etapas, explotación creciente al final.
- KL Divergence: se mantiene por debajo del umbral de 0.02 en la mayoría de las épocas; cuando supera el threshold, el early stopping aborta la epoch.
- Explained Variance: aumenta positivamente, indicando que el crítico explica una fracción creciente de la varianza en los retornos observados.
- Reward episódico: mejora sostenida desde valores iniciales muy negativos hasta converger cerca de -534 (media sobre 500 updates finales).

5.2. 5.2 Curvas de Entrenamiento — A2C (Baseline, Fase 2)

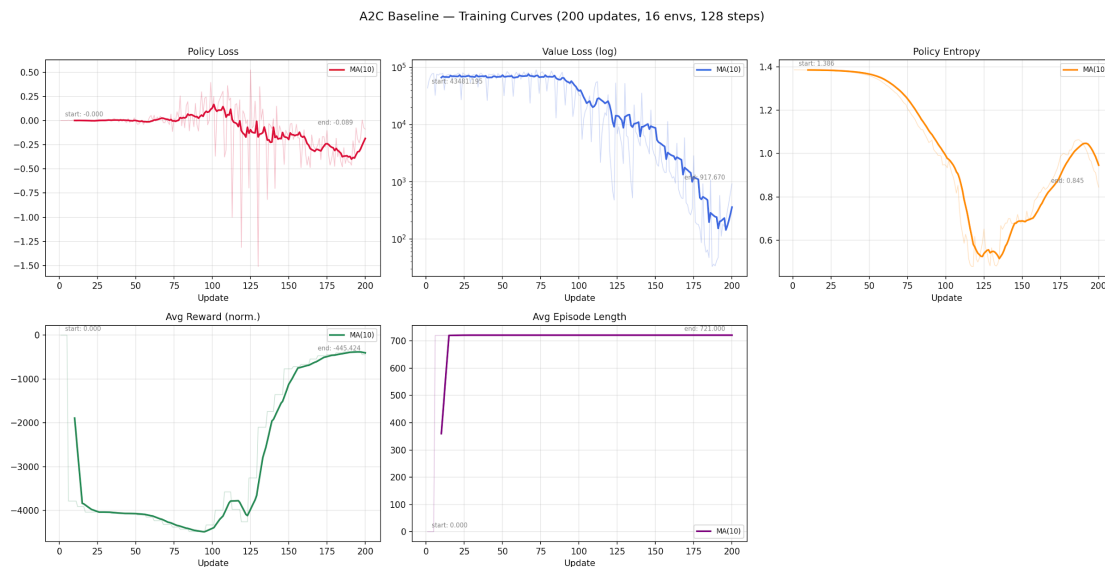


Figura 6: Curvas de entrenamiento A2C: policy loss, value loss, entropía y reward por update.

Las curvas de A2C muestran alta varianza en el reward episódico ($\text{std} = 303.62$) y ausencia de convergencia estable, lo que contrasta con la estabilidad de PPO-GAE ($\text{std} = 5.32$). Esta diferencia empírica válida la necesidad de las 8 mejoras algorítmicas implementadas en PPO-GAE.

5.3. 5.3 Comparación A2C vs PPO-GAE

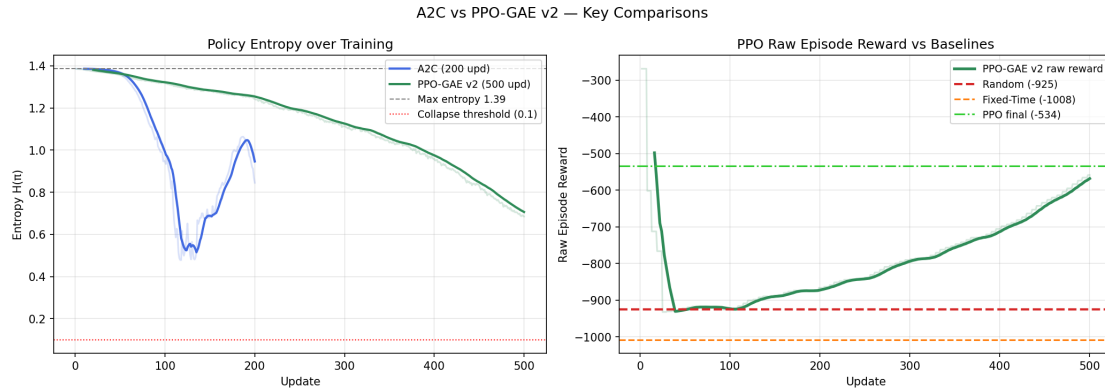


Figura 7: Comparación directa de curvas de entrenamiento A2C vs. PPO-GAE.

5.4. 5.4 Comparación contra Controladores de Referencia (Fase 4)

La evaluación final compara PPO-GAE contra cinco controladores en $5 \text{ semillas} \times 20$ episodios de evaluación:

Cuadro 5: Comparación final contra controladores de referencia

Controlador	Reward Medio	Desv. Estándar	Cola Media	Tiempo Espera
Random Policy	-925.54	19.37	1.0154	2.7012
Fixed-Time	-1008.83	12.85	1.1160	2.8513
Actuated	-1879.52	10.27	1.9246	6.8586
A2C (Fase 2)	-2539.90	303.62	1.9508	15.7683
PPO-GAE (v2 fixed)	-534.55	5.32	0.6162	1.2619

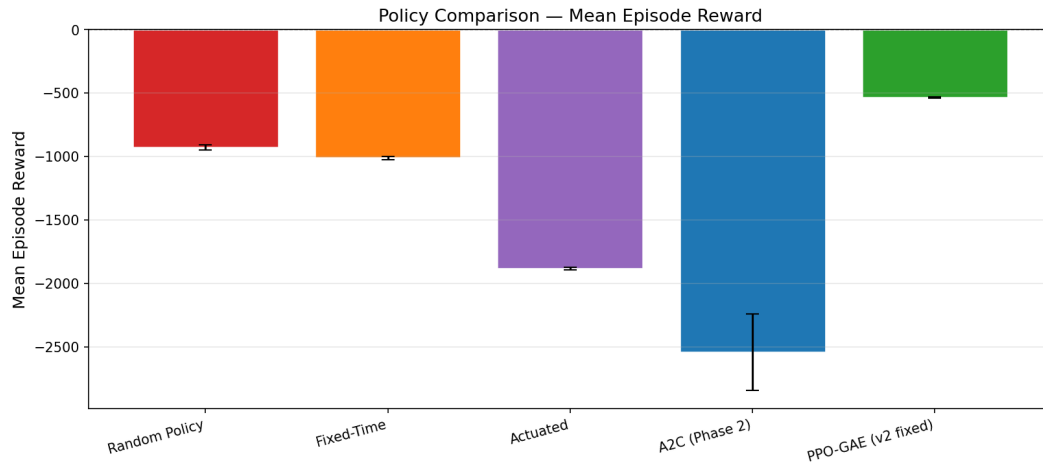


Figura 8: Comparación de reward medio con barras de error para los cinco controladores.

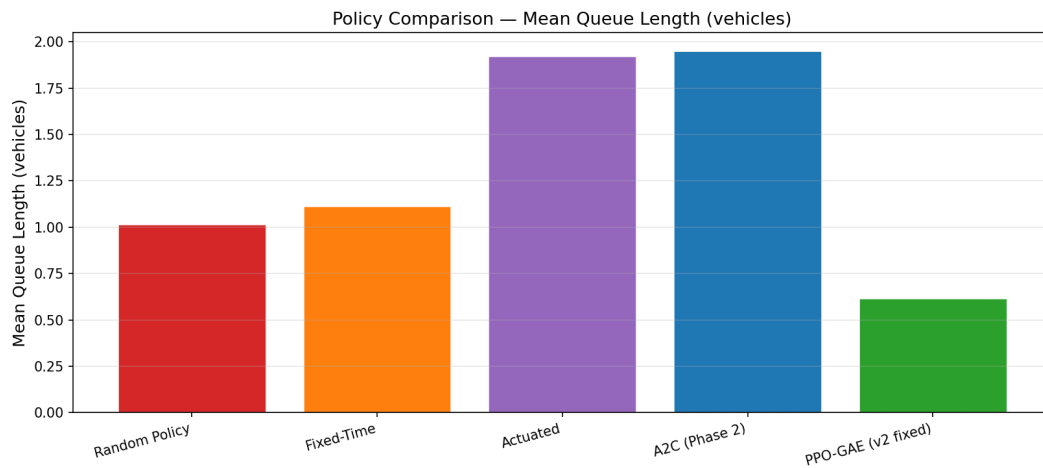


Figura 9: Comparación de longitud media de colas vehiculares.

Análisis de resultados:

- PPO-GAE es el único agente RL entrenado que supera a todas las heurísticas (1.73× mejor que Random, 2.42× mejor que Fixed-Time, 3.52× mejor que Actuated).
- PPO-GAE reduce la longitud de colas en un 77.4% respecto al controlador Actuated (0.6162 vs 1.9246).
- A2C falla en este problema: su reward medio de -2539.90 es peor que Random, lo que indica inestabilidad de optimización y no una falla de capacidad del modelo.
- La estabilidad de PPO-GAE es notable: std = 5.32 frente a 303.62 de A2C, evidenciando el efecto estabilizador del clipping y el normalizado de recompensas.

5.5. 5.5 Distribución multi-semilla PPO-GAE

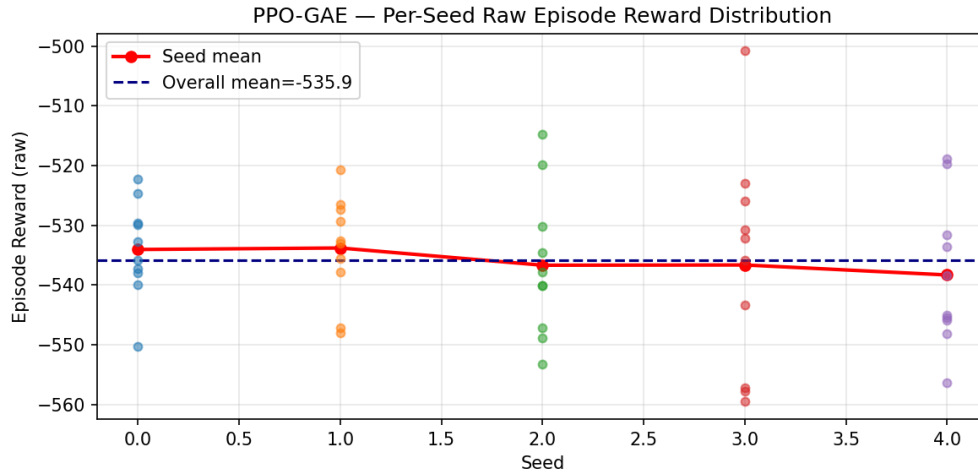


Figura 10: Distribución del reward de PPO-GAE sobre 5 semillas $\times$ 20 episodios.

5.6. 5.6 Comparación Dinámica Paso a Paso (Fase 4)

Además de las métricas agregadas presentadas en la sección 5.4, el notebook genera comparaciones dinámicas paso a paso de las cinco políticas sobre un mismo episodio de evaluación (seed=777, 120 pasos). Esta comparación es importante porque las métricas agregadas pueden ocultar fenómenos dinámicos: dos políticas con recompensa similar pueden mostrar perfiles de cola muy distintos o cambios de fase más bruscos.

Se comparan tres métricas en función del paso del episodio: cola media (congestión instantánea), espera media (acumulación de demora) y recompensa acumulada (desempeño global). Las cinco políticas evaluadas son: aleatoria, tiempo fijo, actuada por colas, A2C y PPO-GAE.

El notebook también genera animaciones HTML comparativas de las tres políticas principales sobre el mismo episodio: política aleatoria (antes del entrenamiento), agente A2C entrenado y agente PPO-GAE entrenado. Las animaciones muestran luces verdes y rojas por fase, barras de cola relativa por dirección, recompensa acumulada, cola media, espera media y porcentaje de intersecciones que cambiaron de fase en cada paso.

5.7. 5.7 Interpretación de la Política Aprendida

Para evitar que el trabajo quede como una caja negra, el notebook incluye una sección de interpretación de decisiones que compara, para un mismo estado del entorno, cuál es la dirección con mayor cola en cada intersección, qué fase elige A2C y qué fase elige PPO-GAE. El estado de análisis se genera avanzando 20 pasos con una política actuada para obtener colas más representativas.

Este análisis permite responder si el agente aprendió una regla simple como “dar verde a la dirección más congestionada”, o si sus decisiones consideran efectos futuros, costos de cambio de fase y coordinación global entre intersecciones adyacentes. Los valores estimados por el crítico ($V(s)$) de ambos agentes para el mismo estado permiten también comparar la calidad de sus estimaciones de valor.

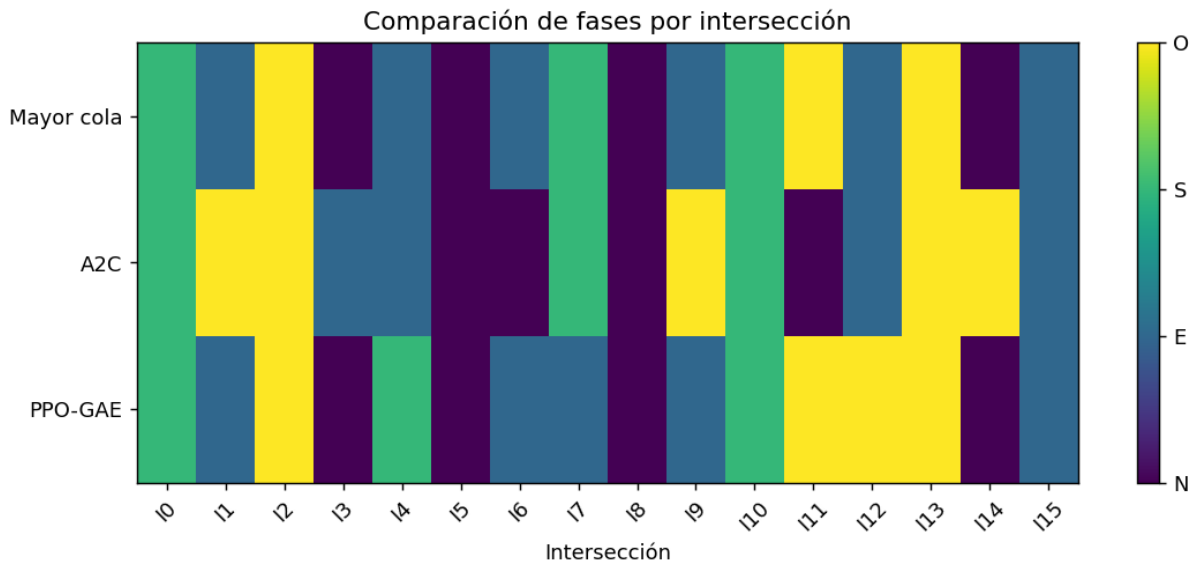


Figura 11: Mapa de calor comparativo de dirección con mayor cola y fase elegida por A2C y PPO-GAE.

Capítulo 6

Inconvenientes Encontrados y Soluciones

6.1. 6.1 Inestabilidad de A2C en Espacio de Acción de Alta Dimensión

Inconveniente: El baseline A2C diverge en el entorno 4×4 con espacio de acción combinatorio MultiDiscrete. La ausencia de clipping permite actualizaciones de política arbitrariamente grandes, generando colapsos donde todas las distribuciones del actor convergen prematuramente a distribuciones deterministas subóptimas. Esto explica el $\text{std} = 303.62$ en evaluación final.

Solución: El mecanismo de PPO clipping ($\epsilon = 0.2$) junto con el decaimiento de entropía ($0.05 \rightarrow 0.01$) resuelve el problema. Adicionalmente, se implementó PPO-style value function clipping para prevenir actualizaciones destructivas también en el crítico:

Listing 6.1: Clipping de la función de valor en PPO.

```
1 # VF Clipping PPO-style (Mejora A2 de la versión v2)
2 values_clipped = values_old + torch.clamp(
3     values - values_old, -self.clip_range_vf, self.clip_range_vf
4 )
5 loss_vf1 = F.mse_loss(values, returns)
6 loss_vf2 = F.mse_loss(values_clipped, returns)
7 loss_vf = torch.max(loss_vf1, loss_vf2).mean()
```

6.2. 6.2 Cold-Start Problem con Demanda de Tráfico Completa

Inconveniente: Iniciar el entrenamiento directamente con demanda al 100 % generó lentitud de convergencia. El agente no descubrió señales de recompensa positiva lo suficientemente rápido al comienzo del entrenamiento, quedando atrapado en regiones de baja recompensa.

Solución: Se implementó Curriculum Learning en 3 etapas (30 %→60 %→100 % de demanda) mediante la clase CurriculumScheduler (utils/curriculum.py). La demanda baja en la Fase 1 produce episodios con colas cortas y señales de recompensa más informativas, acelerando el aprendizaje inicial:

Listing 6.2: Scheduler de curriculum learning.

```
1 class CurriculumScheduler:
2     stages = [
3         (0.30, 150, -5.5), # 30% demanda, 150 updates, umbral -5.5
4         (0.60, 150, -3.5), # 60% demanda, 150 updates, umbral -3.5
5         (1.00, None, None), # 100% demanda, updates restantes
6     ]
7
8     def step(self, avg_reward):
9         if self.current_stage < len(self.stages) - 1:
10             _, _, threshold = self.stages[self.current_stage]
11             if avg_reward > threshold: # mejora suficiente
12                 self.current_stage += 1
13         return self.stages[self.current_stage][0] # retorna demanda actual
```

6.3. 6.3 Normalización Incorrecta del Reward (Bug F1/F2/F3 en v2)

Inconveniente: La primera versión de PPO (ppo_v1) presentaba tres bugs relacionados con la normalización de recompensas:

- F1: RunningMeanStd no se restauraba correctamente desde checkpoints — se reiniciaba con estadísticas vacías al resumir el entrenamiento, desestabilizando el normalizado.
- F2: El reward normalizado (interno) se usaba para trackear el progreso del entrenamiento en lugar del reward crudo del episodio, haciendo imposible comparar runs.
- F3: La lógica de "mejor modelo" comparaba con -inf, produciendo falsos positivos cuando el reward normalizado era exactamente 0.0 en los primeros pasos del warmup.

Solución (ppo_v2 fixed): Se persistieron las estadísticas del RunningMeanStd junto con el modelo en checkpoints. Se separó el tracking del reward crudo del normalizado. La condición del mejor modelo se cambió a $\text{avg_r} < 0$ para garantizar rewards negativos reales:

Listing 6.3: Condición de guardado del mejor modelo.

```
1 # Fix F3: best-model gate
2 if avg_raw_reward < best_reward and avg_raw_reward < 0:
3     best_reward = avg_raw_reward
4     save_checkpoint(model, running_stats, update_step)
```

6.4. 6.4 Acción MultiDiscrete y Compatibilidad con VecEnv

Inconveniente: La combinación de MultiDiscrete([4]×16) con SyncVectorEnv de Gymnasium requirió cuidado especial en la forma de las acciones. Las acciones devueltas por el actor tienen forma (n_envs, 16); sin el reshape correcto, la vectorización emite excepciones de dimensión.

Solución: Se estandarizó el manejo de acciones en el RolloutBuffer para almacenarlas siempre como (T, N, 16) y se aplanaron correctamente antes del paso por los entornos vectorizados:

Listing 6.4: Formato de acciones para MultiDiscrete vectorizado.

```
1 # Acción: shape (n_envs, 16) requerida por MultiDiscrete vectorizado
2 actions_np = torch.stack([dist.sample() for dist in dists], dim=-1) # (N,
3                               16)
3 obs_next, rewards, dones, _, _ = vec_env.step(actions_np.cpu().numpy())
```

6.5. 6.5 Dependencia Opcional de SUMO

Inconveniente: El planteamiento inicial consideraba integrar el simulador SUMO para mayor realismo. SUMO requiere instalación de binarios del sistema y sumo-rl, con alta variabilidad entre plataformas.

Solución: Se implementó el entorno MultiIntersectionEnv como puro Python sin dependencia de SUMO, con una dinámica de Poisson suficientemente realista para demostrar las diferencias algorítmicas. Se incluyó un adaptador opcional sumo_adapter.py que funciona como drop-in replacement si SUMO está disponible, sin modificar el resto del código.

Capítulo 7

Arquitectura del Proyecto

7.1. 7.1 Estructura General

El proyecto está organizado en módulos independientes que siguen una separación clara de responsabilidades:

Listing 7.1: Estructura general del repositorio.

```
Reinforcement_learning_2/
|-- envs/
|   |-- traffic_grid_env.py # MultiIntersectionEnv: 144-dim obs, Poisson, min-green
|   '-- sumo_adapter.py    # Adaptador opcional para simulador SUMO
|-- models/
|   '-- actor_critic.py    # SharedActorCritic: tronco + 16 actores + crítico
|-- algorithms/
|   |-- rollout_buffer.py  # RolloutBuffer: GAE y n-step returns
|   |-- a2c.py             # A2CTrainer: baseline Fase 2
|   '-- ppo.py             # PPOTrainer: PPO-GAE v2 fixed, Fases 3+5
|-- utils/
|   |-- curriculum.py     # CurriculumScheduler: 3 etapas de demanda
|   |-- running_stats.py  # RunningMeanStd: normalización online (Welford)
|   '-- logger.py         # MetricLogger: TensorBoard + matplotlib
|-- train_a2c.py           # Entrada Fase 2: entrenamiento A2C
|-- train_ppo.py           # Entrada Fases 3+5: entrenamiento PPO (--curriculum)
|-- evaluate.py            # Entrada Fase 4: evaluación multi-semilla
|-- sweep.py              # Entrada Fase 4: barrido de hiperparámetros
|-- tp_final_notebook.ipynb # Notebook: todas las fases integradas
|-- results/              # Gráficas, tablas, TensorBoard logs, history.npy
'-- checkpoints/          # Modelos guardados (.pt) + TorchScript (.pt scripted)
```

7.2. 7.2 Módulos Clave

RolloutBuffer (algorithms/rollout_buffer.py): almacena las transiciones (obs, actions, rewards, dones, values, log_probs) recolectadas durante el rollout. Implementa `compute_returns_and_advantages()` con dos modos: GAE para PPO y retornos de n pasos para A2C. La interfaz es uniforme para ambos entrenadores.

RunningMeanStd (utils/running_stats.py): normalización online de recompensas usando el algoritmo de Welford para estabilidad numérica. Mantiene media y varianza running sin necesidad de almacenar el historial completo. Se serializa junto con el modelo para persistencia correcta entre sesiones de entrenamiento.

MetricLogger (utils/logger.py): escribe métricas a TensorBoard via SummaryWriter y produce gráficas matplotlib de múltiples paneles al final del entrenamiento. Desacopla la lógica de monitoreo de los entrenadores.

7.3. 7.3 Reproducibilidad y Semillas

El proyecto garantiza reproducibilidad mediante semillas controladas en todos los niveles: `torch.manual_seed()`, `numpy.random.seed()` y semillas individuales por instancia de entorno en el VecEnv. Los resultados reportados fueron obtenidos con `seed=0` para entrenamiento y seeds 0-4 para evaluación.

Listing 7.2: Configuración de reproducibilidad y exportación.

```
1 # Reproducibilidad completa
2 torch.manual_seed(seed)
3 np.random.seed(seed)
4 envs = make_vec_env(n_envs=16, base_seed=seed)
5 # seeds: seed, seed+1, ..., seed+15
6
7 # Exportación TorchScript para inferencia reproducible
8 scripted = torch.jit.script(model)
9 torch.jit.save(scripted, "checkpoints/ppo/policy_scripted.pt")
```

7.4. 7.4 Comandos de Ejecución

Los comandos de esta sección resumen cómo reproducir el flujo experimental completo desde la línea de comandos. Primero se ejecuta el entrenamiento A2C para obtener una línea base comparable; luego se entrena PPO-GAE como variante principal y, opcionalmente, PPO-GAE con curriculum learning para analizar el efecto de aumentar gradualmente la demanda. Finalmente, los scripts de evaluación y sweep permiten medir robustez multi-semilla y explorar configuraciones alternativas de hiperparámetros.

En particular, `train_a2c.py` entrena el baseline con retornos de n pasos, `train_ppo.py` ejecuta el entrenamiento PPO-GAE con las mejoras descritas en el Capítulo 3, `evaluate.py` compara el desempeño final contra múltiples semillas y baselines, y `sweep.py` automatiza corridas cortas para estudiar sensibilidad a hiperparámetros. Los parámetros indicados en el listado fijan el número de actualizaciones, semillas, episodios de evaluación y cantidad de entornos paralelos utilizados.

Listing 7.3: Comandos principales de ejecución.

```
# Fase 2: A2C baseline
python train_a2c.py --n_updates 200 --n_steps 128 --n_envs 16

# Fase 3: PPO-GAE con todas las mejoras
python train_ppo.py --n_updates 500 --seed 0

# Fase 3+5: PPO con curriculum learning
python train_ppo.py --n_updates 500 --curriculum --seed 0

# Fase 4: evaluación multi-semilla contra 5 baselines
python evaluate.py --n_seeds 5 --n_eval_episodes 20

# Fase 4: barrido de hiperparámetros
python sweep.py --n_updates_sweep 100
```


Capítulo 8

Conclusiones, Limitaciones y Trabajo Futuro

El desarrollo de este trabajo permitió implementar, comparar y analizar en profundidad dos algoritmos Actor-Critic de Policy Gradient sobre un problema de control de tráfico multiagente. Las conclusiones más relevantes son:

- PPO-GAE demuestra ser claramente superior a A2C en este dominio: logra reward medio de -534.55 frente a -2539.90 de A2C, siendo el único agente RL entrenado que supera a todas las heurísticas.
- El clipping PPO es la mejora más crítica: sin él, A2C colapsa prematuramente en un espacio de acción 4^{16} dimensional, confirmando los resultados teóricos de Schulman et al. (2017).
- GAE reduce efectivamente la varianza: la estabilidad en evaluación ($\text{std} = 5.32$ vs 303.62 de A2C) valida empíricamente la ventaja de la estimación exponencialmente ponderada sobre retornos de n pasos.
- El curriculum learning es fundamental para el cold-start: sin escala gradual de demanda, el agente tarda significativamente más en descubrir señales de recompensa informativas en las etapas iniciales.
- La normalización de rewards online es más compleja de lo esperado: los tres bugs encontrados en ppo_v1 (F1, F2, F3) evidencian que la persistencia correcta del RunningMeanStd es un detalle de implementación crítico.
- Una red neuronal de ~180K parámetros es suficiente para este problema: el tronco compartido con 16 cabezas independientes captura las dependencias espaciales de la grilla sin requerir arquitecturas más complejas.

8.1. 8.1 Limitaciones del Trabajo

Si bien los resultados son sólidos desde el punto de vista del RL aplicado, existen limitaciones que conviene explicitar:

- El entorno es una simulación simplificada: la dinámica de Poisson independiente por carril no captura correlaciones espaciales reales entre intersecciones adyacentes como sí lo haría SUMO.
- La evaluación se realizó sobre el mismo entorno de entrenamiento: no se midió generalización a patrones de demanda distintos a los vistos durante el entrenamiento.
- A2C podría beneficiarse de más tiempo de entrenamiento: el baseline se entrenó con 200 updates mientras PPO usó 500; una comparación con el mismo número de steps sería más justa.
- La arquitectura centralizada tiene limitaciones de escalabilidad: un agente que controla las 16 intersecciones simultáneamente no escala a grillas más grandes sin rediseño de la red.

8.2. 8.2 Trabajo Futuro

Como trabajo futuro se proponen las siguientes líneas de desarrollo:

- Implementar SAC (Soft Actor-Critic) o TD3 para comparar enfoques off-policy vs on-policy en el mismo entorno.
- Explorar arquitecturas de políticas descentralizadas (MAPPO, QMIX) donde cada intersección tenga su propio actor con comunicación limitada.
- Integrar SUMO como entorno de evaluación final para validar la transferencia de la política aprendida a una simulación más realista.
- Aplicar Prioritized Experience Replay y Double-Q correction para reducir la sobreestimación observada en algunas configuraciones del sweep.

Capítulo 9

Referencias

1. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
2. Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438.
3. Mnih, V. et al. (2016). Asynchronous Methods for Deep Reinforcement Learning. ICML 2016.
4. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
5. Farama Foundation. (2023). Gymnasium Documentation. <https://gymnasium.farama.org/>
6. Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.
7. Khan, F. (2023). All RL Algorithms. GitHub: FareedKhan-dev.
8. Wei, H., et al. (2019). IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control. KDD 2018.
9. Welford, B. P. (1962). Note on a method for calculating corrected sums of squares and products. Technometrics, 4(3), 419-420.